COOKING IT UP | 1. Administrative and refined proposal

Team: CodeMonks | COP3530 Project 2b | November 8, 2026

DRAFT: Required team details and/or uploaded demo link remain outstanding. See the final page and submission checklist.

Nathan Chin-Lue - GitHub: natecl

Artem Antonian - GitHub: NOT SUPPLIED

Marissa Micioni - GitHub: marissamicioni2025-hub

Repository: https://github.com/natecl/Cooking-It-Up

YouTube demo: Not yet recorded/uploaded. A timed recording script is included in the repository.

Problem and motivation

People with ingredients already available must inspect many recipes to discover what they can cook and what is missing. Cooking It Up turns a pantry list into ranked recipe matches and exposes how two similarity measures differ. The goal is a transparent ingredient-matching comparison, not personalized nutrition or recipe generation.

Implemented features

The C++17 menu accepts ingredient phrases, a positive result count, and Jaccard, cosine, or both. Searches display recipe names and IDs, scores, matched counts, alphabetized missing ingredients, and query time. Users can change settings and search again without restarting; invalid inputs retain the last valid settings. One-shot command-line arguments support repeatable demonstrations.

Empty queries are rejected. Duplicate phrases are removed. Unknown ingredients are reported and retained in score denominators; all-unknown queries return no matches. Only positive-overlap recipes appear. Ties resolve by ascending recipe ID. An original eight-recipe sample makes a fresh checkout runnable without downloading the large dataset.

COOKING IT UP | 2. Data, implementation, and responsibilities

Dataset and cleaning

The source is Food.com Recipes and Interactions, Kaggle version 2 [1], associated with Majumder et al. [2]. RAW_recipes.csv contains 231,637 records. The loader retains 231,636 recipes, exceeding the 100,000 target, and rejects one blank-name record (ID 368257). The resulting vocabulary contains 14,941 normalized ingredient phrases. Interactions, ratings, and user profiles are not used.

The C++ streaming CSV parser handles quoted commas, doubled quotes, and multiline records. A restricted list-of-strings parser reads ingredients without executing data. Cleaning validates positive IDs, rejects empty names/lists, removes duplicate IDs, lowercases ASCII, collapses whitespace, and deduplicates ingredients. Each recipe stores its ID, original name, and sorted unique ingredient IDs. The raw file is downloaded separately; its checksum and rights notice are recorded in the repository.

Algorithms and data structures

A shared unordered_map assigns ingredient IDs; vectors hold the vocabulary and recipes. A two-pointer traversal counts the intersection of sorted ingredient IDs. Jaccard is overlap divided by union cardinality [3]. Binary cosine is overlap divided by sqrt(query size times recipe size) [4]. Both formulas and the overlap traversal are implemented directly, without a similarity library.

A standard-library priority_queue retains the best k lightweight candidates, followed by sorting the retained results. Missing ingredients are computed by another sorted traversal. std::chrono::steady_clock measures elapsed time. CMake builds the program; Python standard-library helpers download and independently validate data. ReportLab generates this PDF. GitHub Actions builds/tests on Linux, macOS, and Windows.

Responsibility and authorship

The proposal assigns Nathan data/Jaccard, Artem cosine/ranking, Marissa interface/benchmarks, and all members integration/report/video. At the user's request, Codex produced the implementation, tests, measurements, technical draft, and recording script. The team had planning materials at the start. Planned roles are not completed-work claims; actual human review and contributions remain to be recorded.

COOKING IT UP | 3. Analysis: changes and worst-case complexity

The implementation preserves the proposal's two algorithms, binary ingredient sets, C++ menu, and large dataset. Added facilities are a bounded heap instead of sorting every recipe, automated tests, deterministic ties, a download helper, one-shot arguments, and a reproducible benchmark mode. These make validation and a clean-checkout demonstration practical. No dense vocabulary-sized recipe vectors are constructed.

Let N be retained recipes, V vocabulary size, M total ingredient memberships, q query cardinality including unknowns, r maximum recipe cardinality, k retained result count capped by N, L maximum string length, T input characters, and B largest CSV record. Bounds below include pathological hash collisions; details are in docs/COMPLEXITY.md.

Major function	Worst-case time	Extra working space
CSV parsing	O(T)	O(B)
Full loading / cleaning	O(T + N r log r L + M V L + N^2)	O(N + M + VL + B), plus names
Query preparation	O(q log q L + q V L)	O(qL)
Sorted overlap	O(q + r)	O(1)
Either score from overlap	O(1)	O(1)
Scan, heap, sort results	O(N(q+r) + N log(k+1) + kL)	O(k), plus returned names
Missing names, one recipe	O(q+r+r log r L)	O(rL)

With ordinary expected hash behavior, loading is O(T + N r log r L). Both similarity methods share the same scan and heap complexity; cosine's square root changes a constant, not Big O. The heap prevents O(N) scored-result storage. Display separately builds an ID hash map: expected O(N) setup, worst-case O(N^2), and O(N) extra storage, excluded from query timing.

Tests cover hand-computed scores, CSV and list parsing, cleaning, empty/unknown inputs, ties, missing ingredients, menu recovery, and independent full-sort rankings. The local suite passes 1,910 C++ checks, CLI integration, and address/undefined-behavior sanitizers. All 14 full-data top-ten rankings/scores agree with an independent Python set-based oracle.

COOKING IT UP | 4. Measured performance and interpretation

Experiments ran on an Apple M4 Pro with 24 GiB RAM, macOS 26.6.2, Apple Clang 21, and a CMake Release build. At each CSV-prefix size, seven fixed queries used k=10, one warmup per method, and 21 timed repetitions with alternating method order. CSV loading is measured separately. Each timing below is the arithmetic mean of the seven query medians; it is not a pooled median.

Recipes	Load (ms)	Jaccard (ms)	Cosine (ms)	Payload MiB
1,000	7.90	0.0078	0.0077	0.13
10,000	78.00	0.1093	0.1027	0.83
100,000	780.01	1.5701	1.5806	6.91
231,636	1875.58	3.8841	3.8815	15.64

The queries span chicken/rice/salt; flour/sugar/eggs/butter; tomato/garlic/oil/onion; milk; an eight-ingredient pantry; chicken plus an unknown; and an all-unknown pantry. Their exact strings, individual medians, nearest-rank p95 values, and ranking agreement appear in benchmarks/results.csv. Search timing includes overlap, scoring, heap selection, final sorting, and selected-name copies; loading, query normalization, missing-name calculation, and terminal output are excluded.

Both methods scale similarly and are close on this machine. At full size their mean-of-medians differs by about 0.003 ms, too small to claim a meaningful universal winner. Six of seven full-size queries have identical ordered top-ten results, including the empty all-unknown result; the eight-ingredient query changes ordering but retains the same ten recipe IDs. Smaller prefixes show more set differences. Prefix composition, cache state, and system activity limit generalization.

Payload is accounted recipe IDs, names, ingredient IDs, vocabulary strings, and hash-key payload; it excludes allocation/capacity/hash-node overhead. Full logical payload is 15.64 MiB. A separate complete load-and-display query peaked at 56.86 MiB process RSS; this includes the runtime and display map and is not per-algorithm working memory. Both score methods use the same O(k) ranking storage.

Correctness also includes a deliberate ranking reversal: for a four-item query, a 2-of-3 match scores Jaccard 0.400 and cosine 0.577; a 4-of-11 match scores 0.364 and 0.603. The metrics express different preferences. Matching ingredients alone does not assess quantities, taste, or suitability.

COOKING IT UP | 5. Reflection, learning, and references

Technical retrospective and group experience

A completed group-experience reflection has not been supplied. This technical retrospective is based on implementation evidence: shared parsing and normalization were central, because every ranking depends on consistent sets. CSV quoting, unknown ingredients, tie handling, and separating load/display time from query time required explicit policies and tests. An initially incorrect ranking-reversal fixture was corrected using hand calculations; the executable then matched the independent oracle.

If restarting the workflow, establish the shared interfaces and small correctness fixtures first, then let each member implement and review their assigned components through real branches and pull requests. Benchmark only a tested Release build. Preserve true contributions rather than generating history after the fact. A future version could add a permanent recipe-ID index, synonyms with explicit evaluation, and more representative shuffled/query workloads. These are improvements, not current features.

Individual learning and outstanding report inputs

Nathan Chin-Lue: Personal learning statement not supplied; the member must review and describe their own work.

Artem Antonian: Personal learning statement not supplied; the member must review and describe their own work.

Marissa Micioni: Personal learning statement not supplied; the member must review and describe their own work.

Before submission: supply Artem's GitHub username, actual human responsibilities and learning/reflection notes, and a real YouTube demo URL. Regenerate this PDF from docs/report_metadata.json. The recording script is prepared; recording and upload remain team tasks. No human experience or individual contribution is inferred from AI-generated code.

References

[1] Shuyang Li. Food.com Recipes and Interactions, Kaggle, version 2. Dataset and original-author rights notice. Accessed November 8, 2026.

[2] Majumder, Li, Ni, and McAuley (2019). Generating Personalized Recipes from Historical User Preferences. EMNLP-IJCNLP. ACL Anthology, D19-1613.

[3] scikit-learn documentation. Jaccard similarity coefficient. Definition reference only; not an implementation dependency.

[4] scikit-learn documentation. Cosine similarity / normalized dot product. Definition reference only.